

Trabalho Prático 2 - Introdução à Inteligência Artificial

Eduardo Santos Birchall

Matrícula: 2024023970

1 Introdução e Objetivo

O trabalho tem como objetivo o estudo e aplicação de técnicas de aprendizado de máquina supervisionado e não-supervisionado. A atividade em questão é a implementação de dois algoritmos, e a comparação de sua performance com a implementação do Scikit-Learn: o K-Nearest Neighbors e o K-Means, quando aplicados ao conjunto de dados NBA Rookie Stats, que contém estatísticas de jogadores novatos da NBA. O processo e metodologia serão explorados nesse documento.

2 Metodologia

2.1 K-Nearest Neighbors

Esse algoritmo de aprendizado supervisionado estima a classe de um ponto descobrindo qual classe compõe a maioria dos pontos em sua volta. Os k pontos mais próximos do ponto escolhido são considerados sua "vizinhança"; a classe mais frequente dentro dessa vizinhança é escolhida como a estimativa da classe do ponto.

A seguir, será explicado o funcionamento da implementação do algoritmo feita nesse trabalho. Dado um dataset de treino T e uma constante k , para se estimar a classe de um ponto p :

1. Para cada ponto t em T , calcula-se a distância euclidiana $D_{p,t}$ entre p e t , definida como $\sqrt{\sum (p_i - t_i)^2}$.
2. Para cada t , adiciona-se, em uma lista L , o par ordenado cujo primeiro valor é $D_{p,t}$ e cujo segundo valor é t .
3. Ordena-se L baseado no primeiro valor de cada par ordenado, obtendo-se uma lista que contém os pontos de T em ordem crescente de distância a p .
4. Entre os k primeiros elementos da lista, escolhe a classe mais frequente. Essa classe será a estimativa.

2.2 K-Means

Esse algoritmo de aprendizado não supervisionado se baseia na escolha aleatória de k pontos no dataset como centroides. Todo ponto p no dataset é atribuído ao cluster cujo centroide é mais próximo a p , e, então, para cada cluster, calcula-se o ponto médio desse cluster. Os pontos médios se tornam os mesmos centroides, e o processo se repete até que convirja.

A implementação do algoritmo foi a seguinte. Dado um dataset X e uma quantidade de agrupamentos k :

1. Inicializa-se o conjunto de centroides $C = \{c_1, c_2, \dots, c_k\}$ escolhendo aleatoriamente k pontos distintos do dataset X .
2. Para cada ponto x em X , calcula-se a distância euclidiana entre x e cada centroide c_j , definida como $\sqrt{\sum (x_i - c_{ji})^2}$. O ponto x é então atribuído ao cluster do centroide mais próximo.
3. Após a atribuição de todos os pontos, recalculam-se os centroides. Cada novo centroide c_j passa a ser a média aritmética das coordenadas de todos os pontos atribuídos ao seu respectivo cluster j .
4. Os passos 2 e 3 são repetidos iterativamente até que a mudança na posição dos centroides entre duas iterações seja menor que uma tolerância pré-definida ($\text{tol} = 10^{-4}$) ou até atingir o limite máximo de iterações.

3 Experimentos e Resultados

Os algoritmos foram rodados junto aos seus equivalentes da biblioteca Scikit-Learn. Essa seção demonstra a performance dos algoritmos implementados, e comparações dessa performance com a dos disponíveis na biblioteca.

3.1 K-Nearest Neighbors

3.1.1 $k = 2$

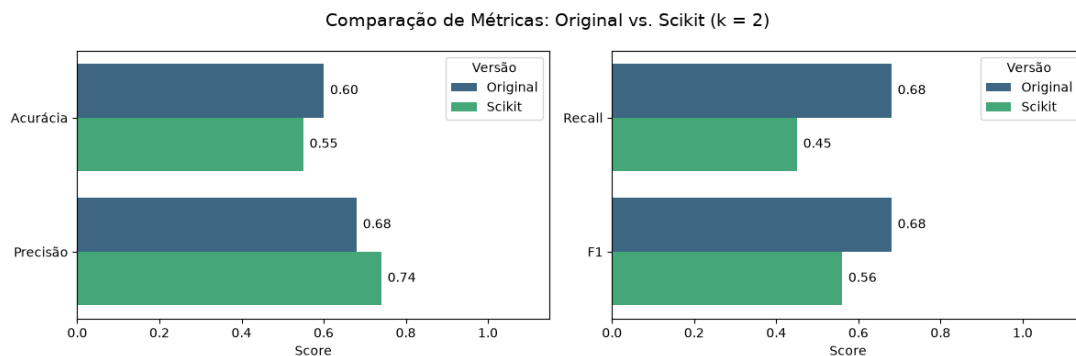


Figure 1: Acurácia, precisão, recall, e F1-score do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 2$.

Nota-se, interessante que a performance do algoritmo original para esse valor de k foi superior à do Scikit-Learn em todas as métricas exceto acurácia.

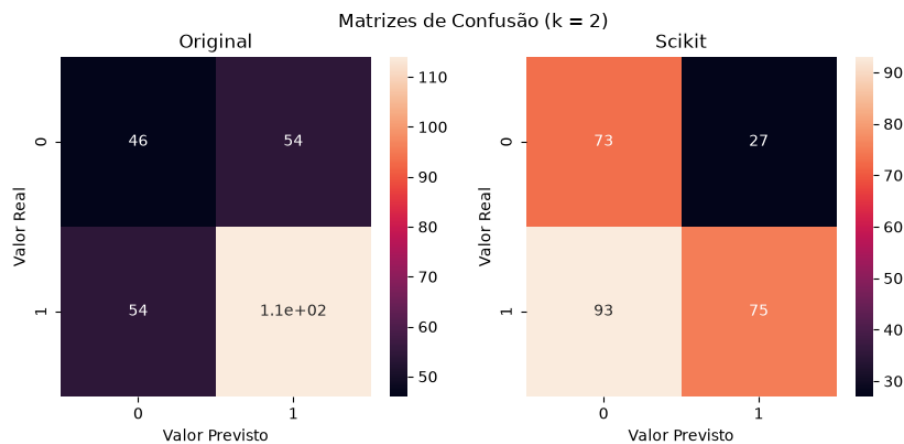


Figure 2: Matriz de confusão do algoritmo original em comparação com o implementado na biblioteca Scikit-Learn quando $k = 2$.

O algoritmo teve uma facilidade muito maior de identificar positivos do que negativos, quando $k = 2$, tendo uma taxa de sucesso maior que a do algoritmo do Scikit-Learn nesse caso.

3.1.2 $k = 10$

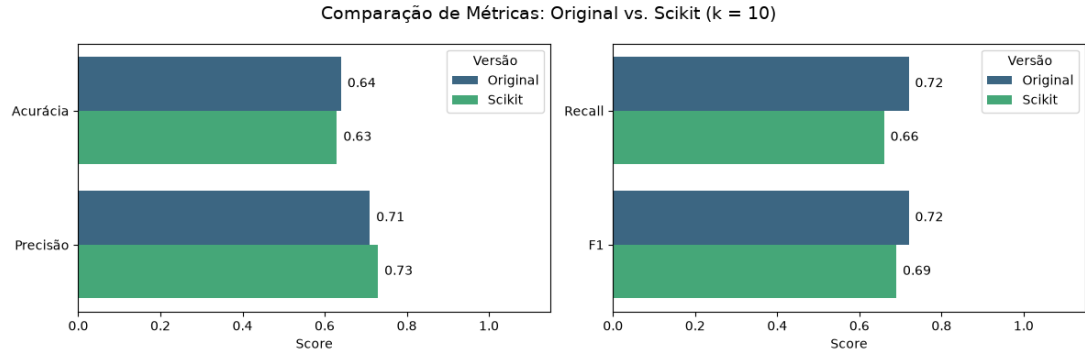


Figure 3: Acurácia, precisão, recall, e F1-score do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 10$.

A implementação do Scikit-Learn tem um aumento comparativo de performance quando k aumenta para 10. A performance absoluta de ambos os algoritmos cresceu, mas a do Scikit-Learn teve um aumento maior que a do algoritmo original.

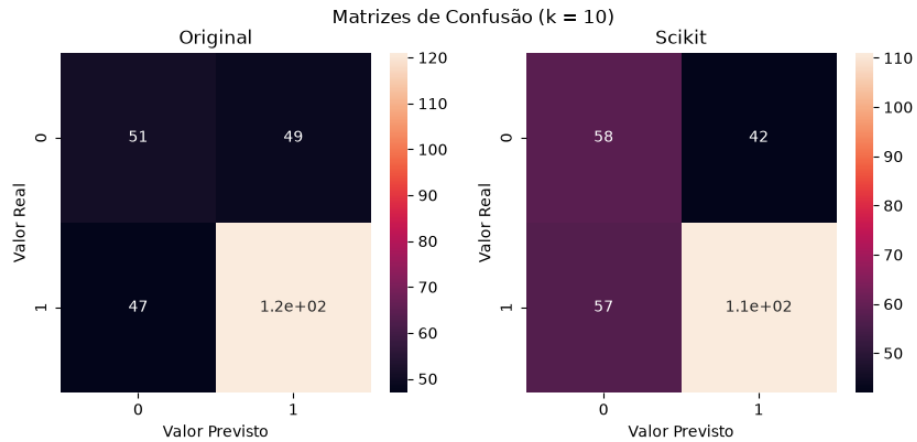


Figure 4: Matriz de confusão do algoritmo original em comparação com o implementado na biblioteca Scikit-Learn quando $k = 10$.

A matriz de confusão do algoritmo original permanece similar, mas a do Scikit-Learn tem uma melhora significativa.

3.1.3 $k = 50$

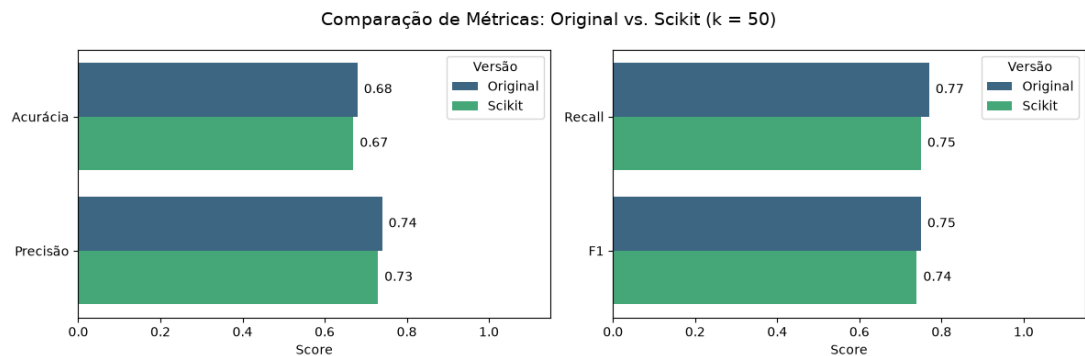


Figure 5: Acurácia, precisão, recall, e F1-score do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 50$.

As performances parecem se tornar mais similares quando k aumenta para 50, ainda tendo aumentos pequenos em termos absolutos.

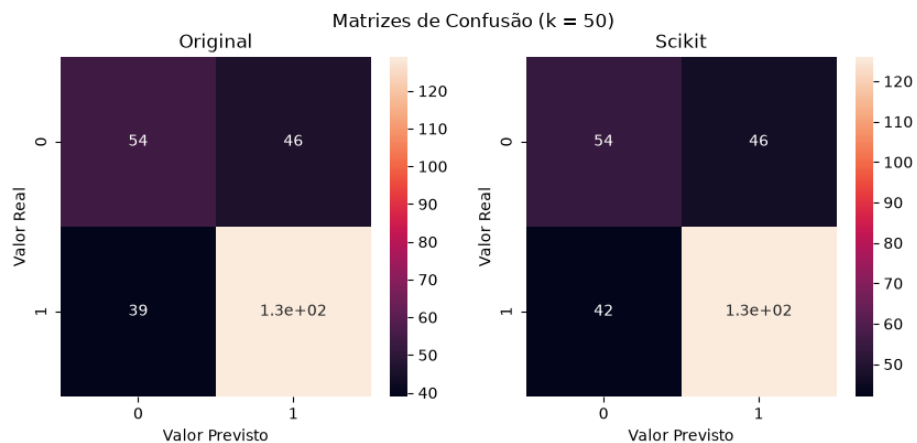


Figure 6: Matriz de confusão do algoritmo original em comparação com o implementado na biblioteca Scikit-Learn quando $k = 50$.

As matrizes de confusão não tiveram mudanças significativas comparado às geradas quando $k = 10$.

3.1.4 $k = 100$

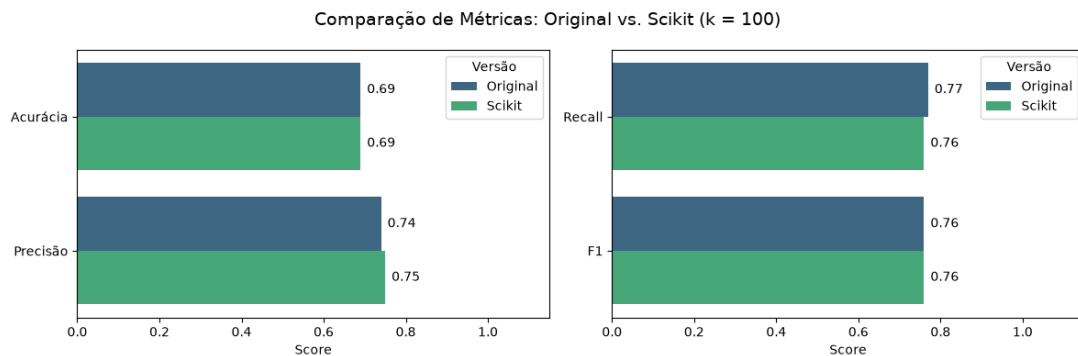


Figure 7: Acurácia, precisão, recall, e F1-score do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 100$.

As performances se tornam quase idênticas quando k chega a esse valor.

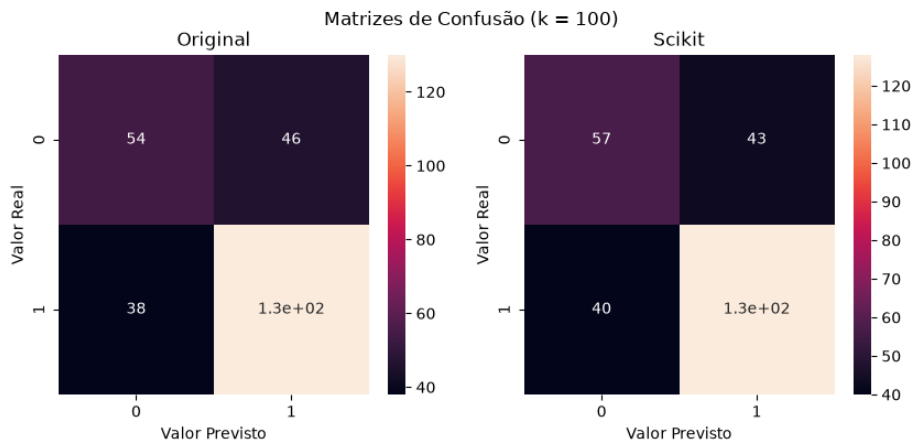


Figure 8: Matriz de confusão do algoritmo original em comparação com o implementado na biblioteca Scikit-Learn quando $k = 100$.

As matrizes de confusão não tiveram mudanças significativas comparado às geradas quando $k = 50$.

3.2 K-Means

3.2.1 $k = 2$

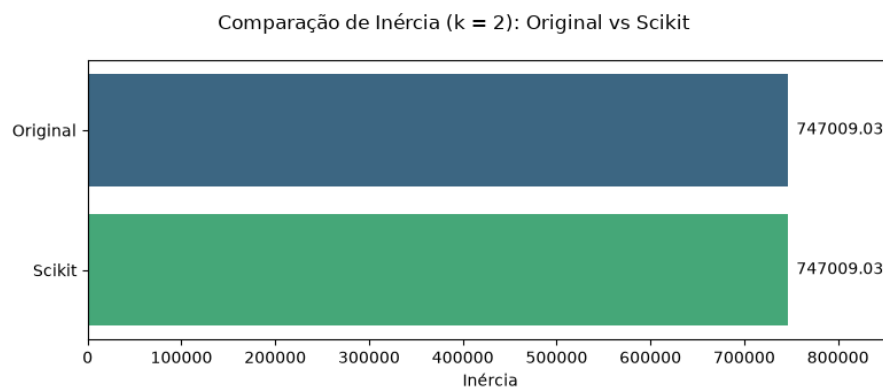


Figure 9: Inércia do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 2$.

As inércias são quase exatamente idênticas quando $k = 2$.

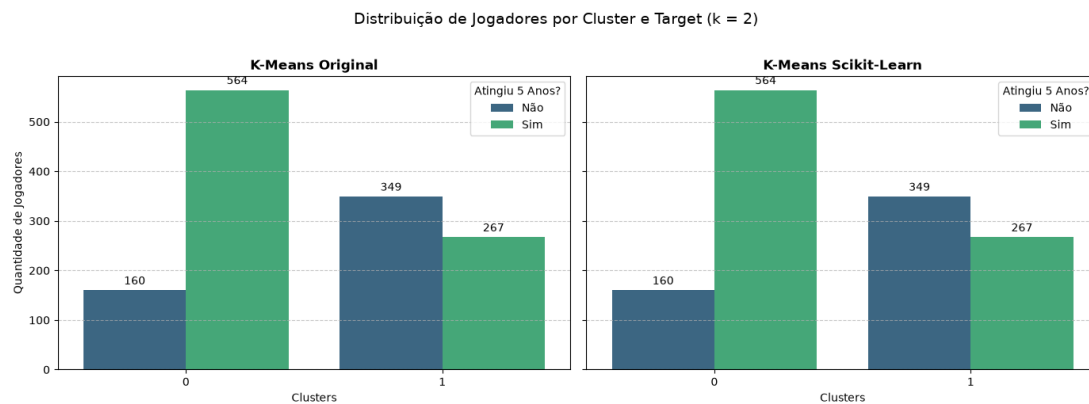


Figure 10: Distribuição da variável alvo entre clusters no algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 2$.

As distribuições são idênticas quando $k = 2$. Nota-se uma discrepância significativa em relação à proporção da variável alvo entre os dois clusters.

3.2.2 $k = 3$

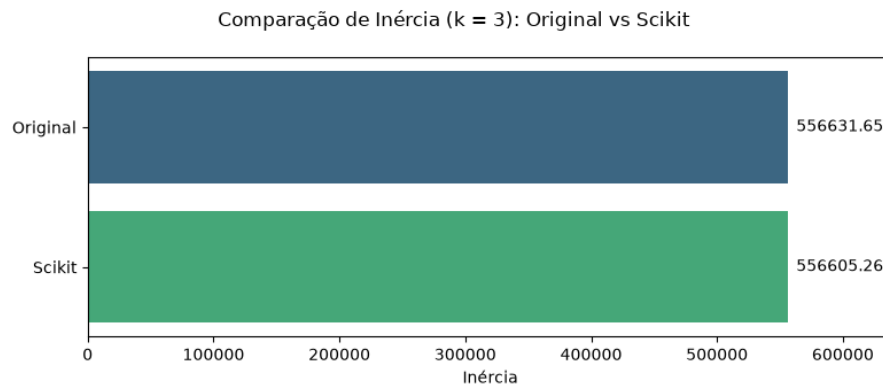


Figure 11: Inércia do algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 3$.

As inércias são quase exatamente idênticas quando $k = 3$, obtendo uma redução em termos absolutos em ambos os casos quando comparado a $k = 2$.

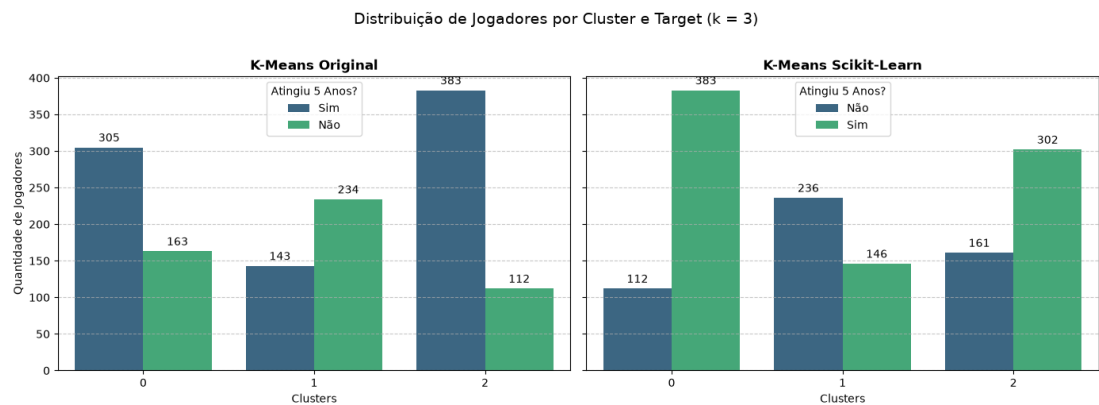


Figure 12: Distribuição da variável alvo entre clusters no algoritmo original (implementado no trabalho) em comparação com o implementado na biblioteca Scikit-Learn quando $k = 3$.

Apesar de terem chegado a distribuições diferentes para seus clusters, os dois algoritmos chegaram a dois agrupamentos que contêm discrepâncias significativas em relação à variável alvo, e um que contém uma distribuição mais igualitária.

4 Discussão

Nota-se que, para o algoritmo K-Nearest Neighbors, o algoritmo implementado no trabalho tem performance superior para valores menores de k , mas que o implementado na biblioteca Scikit-Learn parece ter uma taxa de melhora maior à medida que k aumenta, pelo menos quando se observa até $k = 100$.

O algoritmo K-Means implementado no trabalho obteve uma performance similar à do Scikit-Learn para $k = 2$ e $k = 3$, sugerindo que, para valores pequenos de k em datasets de dimensionalidade similar ao usado nesse trabalho, os dois são comparáveis.

5 Conclusão

A implementação do algoritmo K-Nearest Neighbors mostrou-se dependente da calibração do hiperparâmetro k . O melhor resultado global de classificação foi obtido com valores altos de vizinhos, nos quais o modelo alcançou maior estabilidade preditiva e equilíbrio entre precisão e recall.

Já a implementação do K-Means foi eficaz na identificação de padrões de desempenho sem auxílio de rótulos prévios. A configuração com $k = 3$ trouxe o melhor resultado para o problema, permitindo separar os atletas novatos em três categorias claras (alta probabilidade de permanência, baixa probabilidade e intermediários), fornecendo uma interpretação mais detalhada do dataset do que a divisão puramente binária.